

Mini Project Assessment

Business Case

You are working as a **Data Engineer** for **QuickCart**, a rapidly growing e-commerce and logistics platform.

The company currently faces:

- Slow dashboard performance
- Duplicate customer records
- Delayed reporting
- Pipeline failures
- High storage costs
- Difficulty processing clickstream logs
- Poor query performance on large datasets

The organization wants to build a scalable modern data platform using: Python, SQL, Pandas, , Airflow and modern ETL/ELT architecture.

Section A - Data Engineering Theory

Q1. OLTP vs OLAP Architecture

The company currently stores transactional and analytical workloads in the same SQL database.

Tasks

1. Explain why this creates performance issues.
2. Compare OLTP and OLAP workloads.
3. Suggest an improved architecture.
4. Explain where:
 - PostgreSQL /SQLServer/MySql
 - Data warehouse
 - Data lake

should be used.

Answer -

1. Storing transactional and analytical workloads in the same SQL database can cause performance issues because of these reasons

Storage issue – Database can store upto GB which is ok for OLTP but where as OLAP is a historical data it needs more storage sometimes in TB/PB may which databases don't support which may affect the performance

Transactional performance – OLTP needs fast and safe updates which can be done in Database easily whereas OLAP needs complex reporting and aggregations which may be doing in Database throws performance issues.

2.

OLTP	OLAP
OLTP's are real time transactions	OLAP's are analytical reporting of Historical data
Data is small	Data is Huge
OLTP has Normalized Schema	OLAP has Denormalized Schema
Queries are short and simple like update,insert,delete	Queries are Long and complex like Group by, Join
OLTP is Row Oriented	OLAP is Column oriented
Low Latency	High Latency
Databases - Ex: MySQL,PostgreSQL	DataWarehouses -Ex:Snowflake, BigQuery

3. The Company can use Databases like MySQL,PostgreSQL for OLTP workloads and DataWarehouses like Snowflake and Bigquery for OLAP workloads which improves

Performance.

4. PostgreSQL/SQL Server/MySql - can be used in OLTP workloads

Data warehouse – can be used in OLAP workloads

DataLake – can be used to Ingestion of Raw Data from sales

Q2. ACID vs BASE in Distributed Systems

QuickCart wants to store clickstream events from millions of users.

Tasks

1. Explain why BASE systems are preferred for clickstream systems.
2. Which workloads require ACID compliance?
3. Give examples where eventual consistency is acceptable.

Answer -

1. BASE systems are preferred for clickstream systems because the clickstream events have large data and they can be inconsistent so BASE is a Basically available events are always collected even some events fail, Data may change as new events arrive and BASE supports soft state and BASE are eventually consistent.
2. Customers, orders and product workloads require ACID compliance as ACID supports Atomicity, consistency, isolation and Durability which plays a major role in these workloads
3. In the given input data in Clickstream events the eventual consistency is acceptable as events come in millions per second so they may arrive late or out of order in this case eventual consistency will be accepted. Eventual consistency is accepted in OLAP, Data Lake or warehouse, NoSQL databases and clickstream pipelines

Q3. Storage Formats & Querying

Tasks

1. Compare:
 - CSV
 - JSON
 - Avro
 - Parquet
2. Why do analytical systems prefer columnar storage?
3. Explain avro vs json format.
4. Explain compression benefits in Parquet.

Answer -

1.

CSV	JSON	AVRO	Parquet
Row based	Semi structured	Row based	Column based
Text format - Human Readable	Text Format - Human Readable	Binary + Text format - Machine Readable	Binary Format - Machine Readable
No Schema possible	No schema but structure exists	Schema evolution is supported	Schema supported
Splittability is possible only if CSV is not compressed	Splittability not possible for normal JSON	Splittability is possible	Splittability is possible
Takes large storage since not compressed	Takes large storage	Takes less storage due to compression	Takes less storage due to compression
Useful in small exports	Useful in API's, Logs and semi structured data	Useful in schema evolution, writing heavy workloads	Useful in Analytics and large scale queries

2. Analytical system prefer columnar storage because since it's huge data OLAP and column-based scan is faster than the row-based scan which improves performance

And storage as they scan only required columns and compress the data

3.

JSON	AVRO
Semi structured	Row based
Text Format - Human Readable	Binary + Text format - Machine Readable
No schema but structure exists	Schema evolution is supported
Splittability not possible for normal JSON	Splittability is possible
Takes large storage	Takes less storage due to compression
Useful in API's, Logs and semi structured data	Useful in schema evolution, writing heavy workloads

4. Parquet compresses data upto 90% and stores it as binary which takes very less space compared to any other format and stores it in LookUp table but this format compresses the data only at the storage level and when users want to access the data they can easily decompress it and read and use because of these benefits Parquet format is in high demand in the market.

Q4. Partitioning, Indexing

Tasks

1. Design a partitioning strategy for:
 - orders table
 - clickstream data
2. Suggest indexing strategy for:
 - customer lookups
 - order filtering
 - joins
3. Explain when indexes negatively affect performance.

Answer -

1. For orders table we can use PARTITION for payment_status to make the query faster to find the orders with payment completed or pending or initialized.

Ex: CREATE TABLE orders(

```
Order_id INT PRIMARY KEY,  
Customer_id INT,  
Product_id INT,  
Order_date DATE,  
Quantity INT,  
Unit_price DECIMAL,  
FOREIGN KEY (customer_id) REFERENCES customer(customer_id),  
FOREIGN KEY (product_id) REFERENCES product(product_id),  
PARTITION BY payment_status VARCHAR):
```

For clickstream data we can use PARTITION for event_type so it can have faster query for finding out which event type has more clickstream data

```
EX: CREATE TABLE clickstream(  
    Event_id INT PRIMARY KEY,  
    Customer_id INT,  
    Page_url VARCHAR,  
    Event_timestamp TIMESTAMP,  
    Device_type VARCHAR,  
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id),  
    PARTITION BY event_type VARCHAR):
```

2. Indexing on customer and order lookups , we can use Indexing on customer id if we want to filter the customer table and order table based on customer id , instead of scanning the whole tables for a customer with particular order based on customer id it just scans the customer id column and filters the data. WE can also have Indexing on city, email in customer table and order date and payment status in order table.

```
CREATE INDEX idx_customer_id ON customers(customer_id):  
CREATE INDEX idx_order_customer_id ON orders(customer_id):
```

To create Indexing on Joins we can use it in orders table which joins both customer and product table.

```
CREATE INDEX idx_orders_customer_id ON orders(customer_id):
```

```
CREATE INDEX idx_orders_product_id ON orders(product_id):
```

3. Indexing will not help when the full table aggregation is needed and Indexing will also not help with small data table as index maintenance slows down basic updates like update, delete, insert because every update/insert/delete must also update the index which may affect performance negatively.

Section B - SQL Engineering

Q1. Intermediate SQL Queries

Write SQL queries to:

1. Find top 10 customers by revenue.
2. Find month-over-month sales growth.
3. Find customers who ordered in consecutive months.
4. Find products never ordered.
5. Find revenue contribution percentage by category.

Answer – In the Python script "SQL_assignment_mini_project_1.py" Which is attached along with this document.

Q2. Advanced SQL

Tasks

1. Rank customers based on total revenue.
2. Find running total sales by month.

3. Find the highest selling product per category.
4. Find 7-day rolling average sales.

Answer – In the Python script "SQL_assignment_mini_project_1.py" Which is attached along with this document.

Q3. SQL Optimization Scenario

The following query takes 18 minutes:

```
SELECT c.customer_name,  
       SUM(o.quantity * o.unit_price)  
FROM orders o  
JOIN customers c  
ON o.customer_id = c.customer_id  
WHERE order_date >= '2025-01-01'  
GROUP BY c.customer_name;
```

Tasks

1. Identify possible bottlenecks.
2. Suggest indexes.
3. Explain execution plan analysis.
4. Suggest partitioning improvements.
5. Explain materialized views usefulness.

Answer -

1. Possible bottlenecks: Without indexing whereby clause scans full table for order date and without partitioning order date it is difficult as it contains years of data
2. We can do Indexing on order date and customer_id from customers and orders
3. Execution plan analysis helps in understanding how the database actually executes the SQL query , it shows Table scans,Index usage,Join methods,Sorting operations,Aggregation cost,Estimated cost of each step
4. We can do partition on order date , it helps query scan only partitioned dates

5. Materialized views are helpful because they store pre-computed results of expensive queries, so the database doesn't have to recalculate them every time. They speed up analytics and reporting, especially on large tables and allow you to index the aggregated data, making repeated queries faster.

Q4. Database Design

Design schema for:

1. Shipment tracking
2. Product inventory
3. Customer support tickets

Requirements:

- normalized design
- primary keys
- foreign keys
- indexing recommendations

Answer -

1. Shipment tracking
Customers
Customer_id – PK
Name
Email
Phone

Package
Package_id – PK
shipment_id – FK
Package_type
Weight

Shipment

Shipment_id - PK
Customer_id - FK
Shipment_date
Estimated_delivery_date
Shipment_delivery_address
Shipment_origin_address
Status

Couriers
Courier_id – PK
Shipment_id – FK
Courier_name
Courier_contact_number
Location

Indexing can be done on customer_id, shipment_id, courier_name, location , shipment_date..

2.Product Inventory

Product
Product_id – PK
Product_type
Weight
Batch_number

Inventory
Inventory_id – PK
Product_id – FK
Location_id – FK
Quantity

Location
Location_id - PK
Location_name
address

Indexing can be done on product_id, inventory_id,location_id,batch_number

3.Customer support tickets

Customer

Customer_id –PK

Customer_name

Email

Phone

Tickets

Ticket_id – PK

Customer_id – FK

Ticket_type

Support_channel

Ticket_description

Indexing can be done on customer_id,ticket_id, ticket_type

Section C - Python + Pandas Engineering

Q1. Data Ingestion Pipeline

Using Python and Pandas:

Tasks

1. Read all datasets.
2. Validate schema consistency.

3. Detect duplicate customers.
4. Identify invalid records.

Answer-In the Python script "Python_pandas_data_ingestion_transform_analytics_pipeline.py" Which is attached along with this document.

Q2. Data Cleaning & Transformation

Tasks

1. Standardize city names. (if any)
2. Convert timestamps correctly. (if any)
3. Handle missing values. (if any)
4. Remove corrupted rows.

Answer - In the Python script "Python_pandas_data_ingestion_transform_analytics_pipeline.py" Which is attached along with this document.

Q3. Clickstream Analytics

Using clickstream data:

Tasks

1. Find the most visited pages.
2. Calculate session counts.
3. Find bounce rate.
4. Find mobile vs desktop traffic percentage.

Answer - In the Python script "Python_pandas_data_ingestion_transform_analytics_pipeline.py" Which is attached along with this document.

Q4. Export Optimization

Tasks

1. Export analytical dataset into:
 - o CSV
 - o Parquet
2. Compare storage sizes.
3. Compare read performance.

Answer -

1. In the Python script "Python_pandas_data_ingestion_transform_analytics_pipeline.py" Which is attached along with this document.
2. After exporting the data to csv format the size is 88KB and after exporting data to parquet format the size is 30KB which is a significant difference in storage size
3. After exporting CSV can be read easily where as parquet was not possible to read as it was compressed but if we want to read we can change decompress it and read

Section D - ETL/ELT + Airflow

Q1. ETL vs ELT Architecture

Tasks

1. Compare ETL vs ELT.
2. Explain why cloud warehouses prefer ELT.
3. Design modern ELT pipeline for QuickCart.

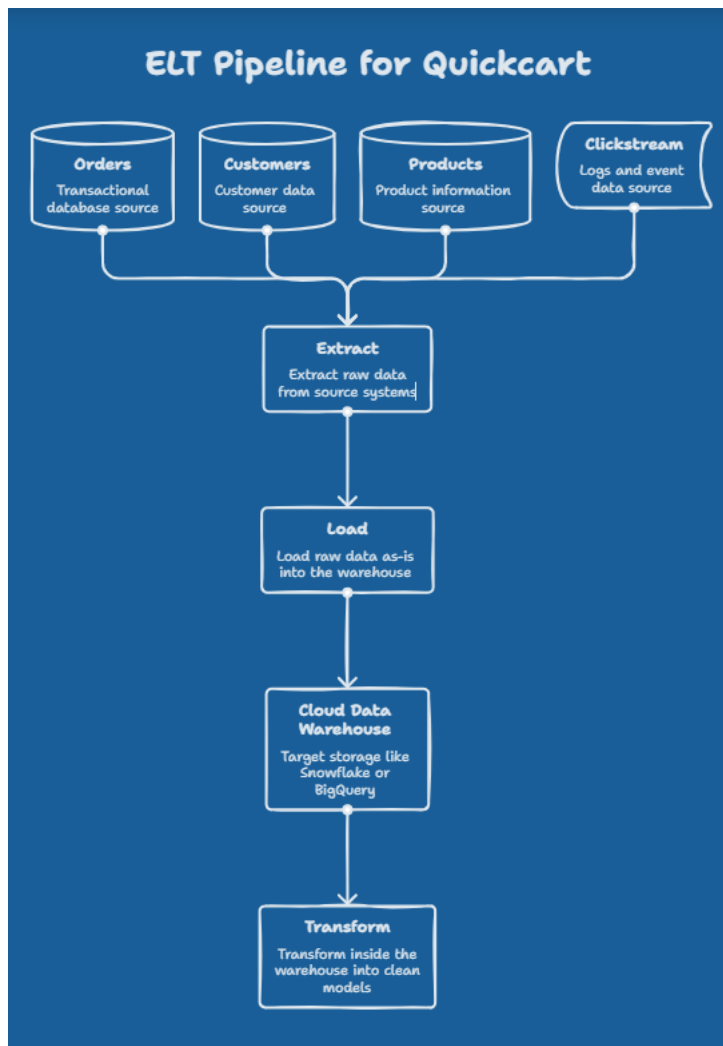
Answer -

1.

ETL	ELT
Extract, Transform , Load	Extract , Load , Transform
ETL is transform before load	ELT is load before transform
Structred data	Can be structured/unstructred data
Transformation will be done before loading into database	Transformation will be done inside database
ETL is used when target system needs cleaned data	ELT is used when target system needs raw data

2.Cloud warehouses prefer ELT because they need raw data for flexible transformation and want to stroe the raw data for future use

3.



Q2. Airflow DAG Development

Create an Airflow DAG that:

Requirements

1. Runs daily at 2 AM
2. Extracts CSV files
3. Performs validation
4. Loads raw data into staging tables
5. Triggers transformation step

6. Sends failure email alerts

DAG must include:

- retries
- retry delays
- task dependencies
- logging
- scheduling
- sensors

Answer - In the Python script "Mni_project_airflow_data_ingestion_dag.py" and "Mini_project_airflow_email_alert_dag.py" Which is attached along with this document.

Q3. Data Warehouse Modeling

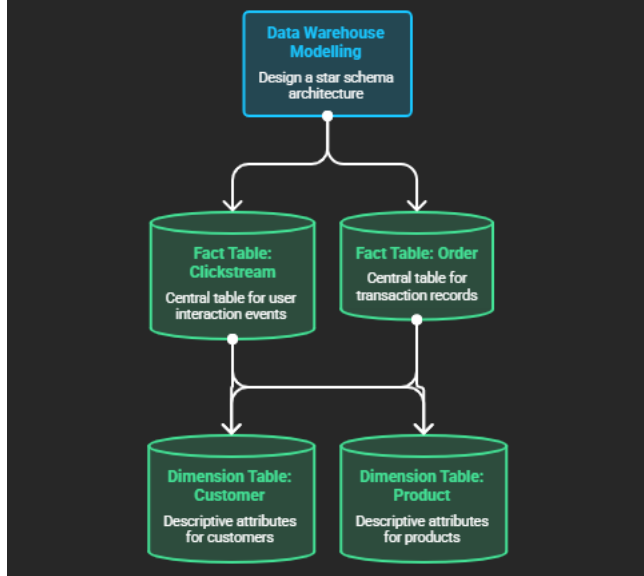
Tasks

1. Design star schema.
2. Identify:
 - fact tables
 - dimension tables

Answer -

1.

Data Warehouse Modelling: Star Schema Design



Dimensions table

Dim Customer (
customer_id (PK)
customer_name
email
city
signup_date
)

Dim Products (
product_id (PK)
product_name
category
supplier_id
cost_price

selling_price
)

Fact Tables

Fact Order (
order_id (PK)
customer_id (FK – Dim Customer)
product_id (FK – Dim Products)
order_date
quantity
unit_price
payment_status

Fact Clickstream (
event_id (PK)
customer_id (FK – Dim Customers)
event_type
page_url
event_timestamp
device_type
)

2.

Fact tables – clickstream , orders
Dimension table – customers ,products

Section E - Performance Engineering Case Study

Scenario

QuickCart processes:

- 50 million clickstream events/day • 5 million orders/day

Problems:

- SQL queries are slow
- Pipelines fail frequently
- Dashboard refresh takes 3 hours • Storage cost is increasing

Tasks

Part 1 - Architecture Design

Design scalable architecture using: • database

- data lake
- warehouse
- orchestration
- transformation layer

Part 2 - Optimization

Explain:

1. Partitioning strategy
2. Indexing strategy
3. Compression strategy
4. Query optimization

5. Incremental loading
6. Columnar storage benefits

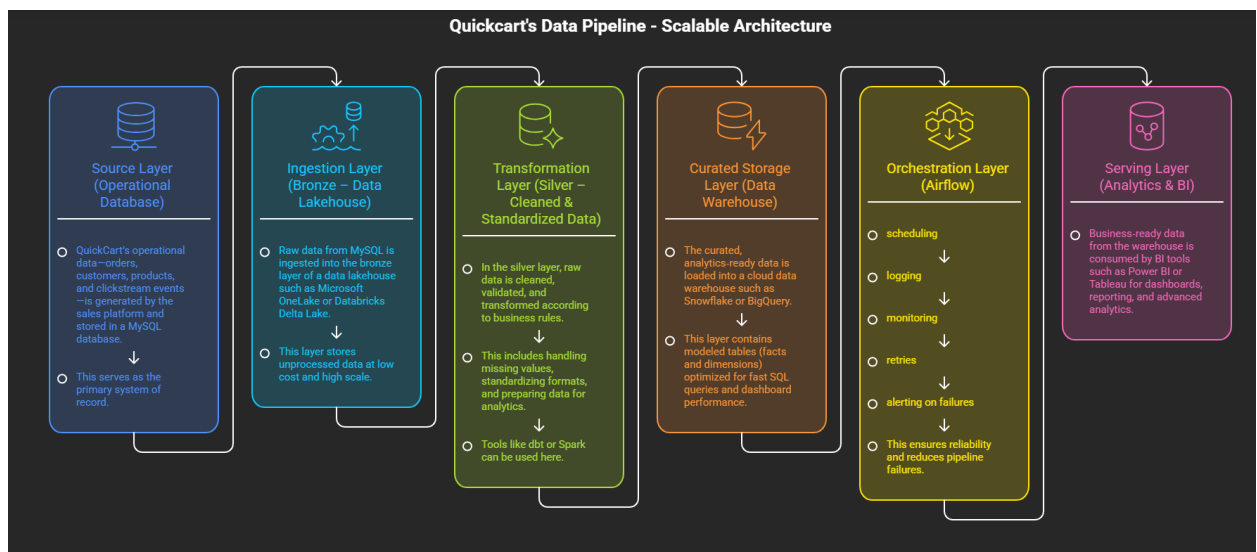
Part 3 - Reliability

Explain:

1. Retry handling
2. Failure alerts
3. Data quality validation

Answer -

Part 1



QuickCart Data Pipeline – Scalable Architecture Overview

1. Source Layer (Operational Database)

QuickCart's operational data—orders, customers, products, and clickstream events—is generated by the sales platform and stored in a MySQL database. This serves as the primary system of record.

2. Ingestion Layer (Bronze – Data Lakehouse)

Raw data from MySQL is ingested into the **bronze layer** of a data lakehouse such as **Microsoft OneLake** or **Databricks Delta Lake**. This layer stores unprocessed data at low cost and high scale.

3. Transformation Layer (Silver – Cleaned & Standardized Data)

In the **silver layer**, raw data is cleaned, validated, and transformed according to business rules. This includes handling missing values, standardizing formats, and preparing data for analytics. Tools like **dbt** or **Spark** can be used here.

4. Curated Storage Layer (Data Warehouse)

The curated, analytics-ready data is loaded into a cloud data warehouse such as **Snowflake** or **BigQuery**. This layer contains modeled tables (facts and dimensions) optimized for fast SQL queries and dashboard performance.

5. Orchestration Layer (Airflow)

An orchestration system like **Apache Airflow** manages the entire pipeline:

- scheduling
- logging
- monitoring
- retries
- alerting on failures This ensures reliability and reduces pipeline failures.

6. Serving Layer (Analytics & BI)

Business-ready data from the warehouse is consumed by BI tools such as **Power BI** or **Tableau** for dashboards, reporting, and advanced analytics.

Part 2

1. Partitioning Strategy Partition large fact tables (clickstream, orders) by date or customer_id to reduce scan size. This ensures queries only read relevant partitions, improving performance and lowering compute cost and makes the queries faster.

2. **Indexing Strategy** Create indexes on high-filter columns like `customer_id`, `product_id`, and `event_timestamp`. This speeds up lookups and joins, instead of scanning the full table the query only scans indexed columns.
3. **Compression Strategy** Use columnar compression to reduce storage footprint and improve I/O efficiency. Compression works extremely well on repetitive clickstream and order data.
4. **Query Optimization** Optimize queries by selecting only required columns, using partition pruning, Indexing and avoiding unnecessary joins.
5. **Incremental Loading** Load only new or changed records (CDC) instead of full reloads. This reduces pipeline runtime, prevents failures, and keeps daily ingestion manageable.
6. **Columnar Storage Benefits** Columnar formats (Parquet, ORC) allow faster analytical queries by scanning only needed columns. They also provide better compression and significantly lower storage and compute cost.

Part 3

1. **Retry Handling** Retries allow failed tasks to automatically run again after a short delay, reducing pipeline interruptions caused by temporary issues (network, API limits, cluster load). Airflow lets you configure retry count and backoff to improve reliability.
2. **Failure Alerts** Failure alerts notify the data team immediately when a task or DAG fails. Airflow supports email, Slack, Teams so issues can be investigated quickly before they impact dashboards or downstream jobs.
3. **Data Quality Validation** Data quality checks ensure incoming data meets expected rules (no nulls, valid ranges, correct formats). Validations catch bad data early, preventing corrupted datasets from reaching the warehouse or breaking analytics.

